

Language Model in Information Retrieval

Introduction to Information Retrieval

Dwaipayan Roy

Department of Computational and Data Sciences



What is Language Model

- Probability distribution over strings of text:

- ▶ how likely is the occurrence of a given string (**observation**) in a given **language** (**proba. distribution**).

- ★ $p_1 = P(\text{a quick brown dog})$

- ★ $p_2 = P(\text{dog quick a brown})$

- ★ $p_3 = P(\text{un chien brun rapide})$

- ★ $p_4 = P(\text{un brown dog rapide})$

What is Language Model

- Probability distribution over strings of text:

- ▶ how likely is the occurrence of a given string (**observation**) in a given **language** (**proba. distribution**).

- ★ $p_1 = P(\text{a quick brown dog})$
 - ★ $p_2 = P(\text{dog quick a brown})$
 - ★ $p_3 = P(\text{un chien brun rapide})$
 - ★ $p_4 = P(\text{un brown dog rapide})$
 - ★ For **English**:

What is Language Model

- Probability distribution over strings of text:

- ▶ how likely is the occurrence of a given string (**observation**) in a given **language** (**proba. distribution**).

- ★ $p_1 = P(\text{a quick brown dog})$

- ★ $p_2 = P(\text{dog quick a brown})$

- ★ $p_3 = P(\text{un chien brun rapide})$

- ★ $p_4 = P(\text{un brown dog rapide})$

- ★ For **English**: $p_1 > p_2 > p_4 > p_3$

What is Language Model

- Probability distribution over strings of text:

- ▶ how likely is the occurrence of a given string (**observation**) in a given **language** (**proba. distribution**).
 - ★ $p_1 = P(\text{a quick brown dog})$
 - ★ $p_2 = P(\text{dog quick a brown})$
 - ★ $p_3 = P(\text{un chien brun rapide})$
 - ★ $p_4 = P(\text{un brown dog rapide})$
 - ★ For **English**: $p_1 > p_2 > p_4 > p_3$
 - ★ In most of IR, assume that $p_1 == p_2 \rightarrow$ term ordering ignored.

What is Language Model

- Probability distribution over strings of text:
 - ▶ how likely is the occurrence of a given string (**observation**) in a given **language** (**proba. distribution**).
 - ★ $p_1 = P(\text{a quick brown dog})$
 - ★ $p_2 = P(\text{dog quick a brown})$
 - ★ $p_3 = P(\text{un chien brun rapide})$
 - ★ $p_4 = P(\text{un brown dog rapide})$
 - ★ For **English**: $p_1 > p_2 > p_4 > p_3$
 - ★ In most of IR, assume that $p_1 == p_2 \rightarrow$ term ordering ignored.

Language Model

- ▶ Language model assigns a probability to a sequence of words by means of a probability distribution.

What is Language Model

- Probability distribution over strings of text:
 - ▶ how likely is the occurrence of a given string (**observation**) in a given **language** (**proba. distribution**).
 - ★ $p_1 = P(\text{a quick brown dog})$
 - ★ $p_2 = P(\text{dog quick a brown})$
 - ★ $p_3 = P(\text{un chien brun rapide})$
 - ★ $p_4 = P(\text{un brown dog rapide})$
 - ★ For **English**: $p_1 > p_2 > p_4 > p_3$
 - ★ In most of IR, assume that $p_1 == p_2 \rightarrow$ term ordering ignored.

Language Model

- ▶ Language model assigns a probability to a sequence of words by means of a probability distribution.

How LM can be used for Retrieval?

Noisy-Channel Model of IR

- IR $\rightarrow$ transforming information need into keyword query before performing a retrieval.

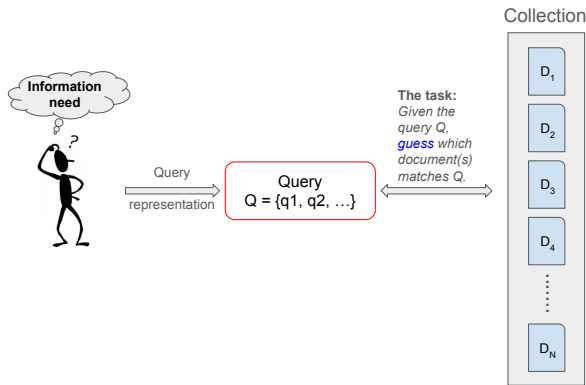
Noisy-Channel Model of IR

- IR $\rightarrow$ **transforming** information need into keyword query before performing a retrieval.
- What if the transformation is noisy?

Noisy-Channel Model of IR

- IR $\rightarrow$ **transforming** information need into keyword query before performing a retrieval.
- What if the transformation is noisy?

most **prestigious** award in science in India equivalent to Nobel

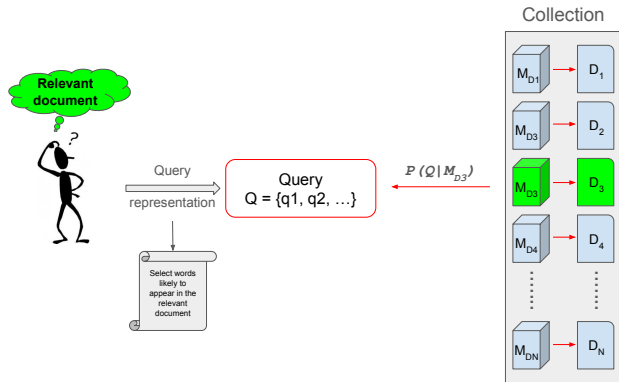


Retrieval based on Language Model

- A document is a good match to a query if the *document model* is likely to generate the query.

Retrieval based on Language Model

- A document is a good match to a query if the *document model* is likely to generate the query.



Retrieval based on Language Model

- Coming up with a *good query* (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*

Retrieval based on Language Model

- Coming up with a **good query** (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*
- The language modeling approach to IR directly models this idea:
 - ▶ a document is a good match to a query if the *document model* is likely to generate the query;
- Build a probabilistic language model M_d from each document d .

Retrieval based on Language Model

- Coming up with a **good query** (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*
- The language modeling approach to IR directly models this idea:
 - ▶ a document is a good match to a query if the *document model* is likely to generate the query;
- Build a probabilistic language model M_d from each document d . (**document model**)

Retrieval based on Language Model

- Coming up with a **good query** (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*
- The language modeling approach to IR directly models this idea:
 - ▶ a document is a good match to a query if the *document model* is likely to generate the query;
- Build a probabilistic language model M_d from each document d . (**document model**)
- Build a probabilistic language model M_Q from the query Q .

Retrieval based on Language Model

- Coming up with a **good query** (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*
- The language modeling approach to IR directly models this idea:
 - ▶ a document is a good match to a query if the *document model* is likely to generate the query;
- Build a probabilistic language model M_d from each document d . (**document model**)
- Build a probabilistic language model M_Q from the query Q . (**query model**)

Retrieval based on Language Model

- Coming up with a **good query** (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*
- The language modeling approach to IR directly models this idea:
 - ▶ a document is a good match to a query if the *document model* is likely to generate the query;
- Build a probabilistic language model M_d from each document d . (**document model**)
- Build a probabilistic language model M_Q from the query Q . (**query model**)
- Rank documents based on the probability of document model generating the query: $P(M_Q|M_d)$.

Retrieval based on Language Model

- Coming up with a **good query** (from user's perspective):
 - ▶ *Use words that would likely appear in relevant documents as the query terms.*
- The language modeling approach to IR directly models this idea:
 - ▶ a document is a good match to a query if the *document model* is likely to generate the query;
- Build a probabilistic language model M_d from each document d . (**document model**)
- Build a probabilistic language model M_Q from the query Q . (**query model**)
- Rank documents based on the probability of document model generating the query: $P(M_Q|M_d)$.

will be denoting as $P(Q|M_d)$

Retrieval based on Language Model

the basic steps

- 1 *Construct language model* M_d associated with each document d of the collection.

Retrieval based on Language Model

the basic steps

- ① *Construct language model* M_d associated with each document d of the collection.
- ② Given query Q , *compute the generation probability* of Q from each of M_d s $\rightarrow P(Q|M_d)$.

Retrieval based on Language Model

the basic steps

- 1 *Construct language model* M_d associated with each document d of the collection.
- 2 Given query Q , *compute the generation probability* of Q from each of M_d s $\rightarrow P(Q|M_d)$.
- 3 *Rank the documents* based on the probability in decreasing order.

Brief recap

- Probability
- Probability distribution
- Discrete random variable

Discrete Random Variable

- **A** is a discrete random variable if:
 - ▶ **A** describes an event whose outcome has some degree of uncertainty (this property makes the variable *random*);
 - ▶ **A** describes an event with a finite number of possible outcomes (this property makes the random variable *discrete*).

Discrete Random Variable

Example

A random variable is called **discrete** if it has either **a finite** or **a countable** number of possible values.

- **A** = *it will rain tomorrow*
- **A** = *the coin-flip will show heads*
- **A** = *you will win the lottery in your lifetime*
- **A** = *Virat will retire*
- **A** = *number of goals that will be scored by Lio Messi in the next match*

Probability Distribution

- A probability distribution gives the probability of each possible outcome of a random variable.



- $P(\text{red})$ = probability of drawing a red ball from the urn
- $P(\text{blue})$ = probability of drawing a blue ball from the urn
- $P(\text{orange})$ = probability of drawing an orange ball from the urn

Probability Distribution

- Two necessary condition for being a *Probability Distribution*:

- ① probability assigned to each possible outcome must be between 0 and 1 (inclusive)

$$0 \leq P(\text{red}) \leq 1 \quad 0 \leq P(\text{blue}) \leq 1 \quad 0 \leq P(\text{orange}) \leq 1$$

- ② sum of probabilities across outcomes must be 1

$$P(\text{red}) + P(\text{blue}) + P(\text{orange}) = 1$$

Estimating a Probability Distribution

example



- $P(\text{Red}) = ?$
- $P(\text{Blue}) = ?$
- $P(\text{Orange}) = ?$

Estimating a Probability Distribution

example



- $P(\text{red}) = 10/20 = 0.5$
- $P(\text{blue}) = 5/20 = 0.25$
- $P(\text{yellow}) = 5/20 = 0.25$
- Note: balls are drawn with replacement into the urn (putting it back in the urn before taking another one out)

Estimating a Probability Distribution

example



- $P(\text{Red}) = 0.5$
- $P(\text{Blue}) = 0.25$
- $P(\text{Orange}) = 0.25$
- $P(\text{Orange Orange Orange}) = ?$
- $P(\text{Orange Blue Orange}) = ?$
- $P(\text{Orange Red Blue Orange}) = ?$

Document as a Probability Distribution



- Each colored ball = a distinct term in the document.
- ● → The
- ● → quick
- ● → brown
- ...

Document as a Probability Distribution



- Each colored ball = a distinct term in the document.
- ● → The
- ● → quick
- ● → brown
- ...

Probability distribution of doc $d \rightarrow$ Language Model of d

Language Model

Language Model

Assigns a probability to a **sequence of words** by means of a probability distribution.

Language Model

Language Model

Assigns a probability to a **sequence of words** by means of a probability distribution.

- Any span of text can be used to estimate a language model:
 - ▶ a sentence
 - ★ a **document** - document language model.
 - ★ a **query** - query language model.
 - ★ a **corpus** - collection language model.
 - ▶ the entire web

Language Model

Language Model

Assigns a probability to a **sequence of words** by means of a probability distribution.

- Any span of text can be used to estimate a language model:
 - ▶ a sentence
 - ★ a **document** - document language model.
 - ★ a **query** - query language model.
 - ★ a **corpus** - collection language model.
 - ▶ the entire web
- Given a language model, we can assign a probability to any span of text:

Language Model

Language Model

Assigns a probability to a **sequence of words** by means of a probability distribution.

- Any span of text can be used to estimate a language model:
 - ▶ a sentence
 - ★ a **document** - document language model.
 - ★ a **query** - query language model.
 - ★ a **corpus** - collection language model.
 - ▶ the entire web
- Given a language model, we can assign a probability to any span of text:
 - ▶ how likely that a given text would be generated from that language model.

Constructing the Language Model of a Document

- Unigram: [All], [men], [must], [serve], ...
- Bigram: [All men], [men must], [must serve], ...
- Unigram + Bigram: [All], [men], [must], [serve], ... [All men], [men must], [must serve], ...
- ...

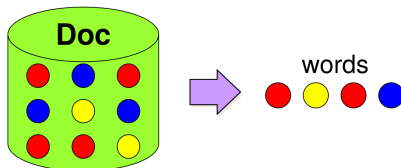
Unigram Language Model

Definition

- A **unigram language model** assumes that each word in a sequence (e.g., a query or document) is generated **independently** of all other words.

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

- Terms are randomly drawn from a document (with replacement).



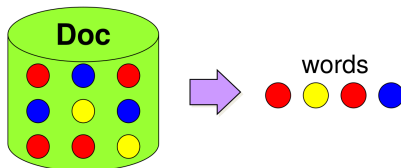
Unigram Language Model

Definition

- A **unigram language model** assumes that each word in a sequence (e.g., a query or document) is generated **independently** of all other words.

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

- Terms are randomly drawn from a document (with replacement).



- $P(\text{red, yellow, red, blue}) = P(\text{red}) \times P(\text{yellow}) \times P(\text{red}) \times P(\text{blue}) = 4/9 \times 2/9 \times 4/9 \times 2/9$

Unigram Language Model

- **Unigram** → because we estimate (and predict) the likelihood of each word *independent* of any other word.
- Assumes that words are independent: (not always correct!)
 - ▶ probability of seeing **Kong** is the same, even if the previously sampled word is **Hong**.
- Other language models (bigram, trigram etc.) take context into account.
 - ▶ works better for applications like *speech recognition* or *automatic language translation*.
- **Unigram models work well for information retrieval.**

Unigram Language Model

- **Unigram** → because we estimate (and predict) the likelihood of each word *independent* of any other word.
- Assumes that words are independent: (not always correct!)
 - ▶ probability of seeing **Kong** is the same, even if the previously sampled word is **Hong**.
- Other language models (bigram, trigram etc.) take context into account.
 - ▶ works better for applications like *speech recognition* or *automatic language translation*.
- **Unigram models work well for information retrieval.**
- **Will be estimated during the indexing.**

Unigram Language Model

estimation

- General estimation approach:
 - ▶ casfold, tokenize/split the text into terms;
 - ▶ count the total number of term occurrences ($|D|$);
 - ▶ count the number of occurrences of each term ($tf(t, D)$).

Unigram Language Model

estimation

- General estimation approach:

- ▶ casefold, tokenize/split the text into terms;
- ▶ count the total number of term occurrences ($|D|$);
- ▶ count the number of occurrences of each term ($tf(t, D)$).
- ▶ Assign term t a probability equal to:

$$P(t) = \begin{cases} \frac{tf(t,D)}{|D|} & \text{when span of text is document} \\ \frac{cf(t)}{|V|} & \text{when span of text is collection / corpus} \\ \frac{tf(t,Q)}{|Q|} & \text{when span of text is query} \end{cases}$$

Language Model for Information Retrieval

$$P(t) = \begin{cases} \frac{tf(t,D)}{|D|} & \text{when span of text is document} \\ \frac{cf(t)}{|V|} & \text{when span of text is collection / corpus} \\ \frac{tf(t,Q)}{|Q|} & \text{when span of text is query} \end{cases}$$

- A language model is a probability distribution defined over a particular vocabulary.
- In this analogy, each color represents a vocabulary term and each ball represents a term occurrence in the text used to estimate the language model.



Document Language Model

- The language model estimated from document D is sometimes denoted as:

$$\theta_D \text{ OR } M_D$$

- The probability assigned to term t by the language model estimated from document D denoted as:

$$P(t|D) = P(t|M_D) = \frac{tf(t, D)}{|D|}$$

Document Language Model

Artificial intelligence is the future of technology. Technology is evolving rapidly, and artificial intelligence is transforming industries.

- Make the unigram language model of the document.

Document Language Model

Artificial intelligence is the future of technology. Technology is evolving rapidly, and artificial intelligence is transforming industries.

- Make the unigram language model of the document.
- What about bigram language model!?

Document Language Model

- Suppose we have a document D , with language model M_D .
- We can use this language model to determine:
 - ▶ the probability of a term t . **How?**

Document Language Model

- Suppose we have a document D , with language model M_D .
- We can use this language model to determine:
 - ▶ the probability of a term t . **How?**
 - $P(t|d) = \frac{\text{count}(t,d)}{\text{doc. length}}.$

Document Language Model

- Suppose we have a document D , with language model M_D .
- We can use this language model to determine:
 - ▶ the probability of a term t . **How?**
 - $P(t|d) = \frac{\text{count}(t,d)}{\text{doc. length}}$.
 - ▶ the probability of a particular sequence of text. **How?**

Document Language Model

- Suppose we have a document D , with language model M_D .
- We can use this language model to determine:
 - ▶ the probability of a term t . **How?**
 - $P(t|d) = \frac{\text{count}(t,d)}{\text{doc. length}}$.
 - ▶ the probability of a particular sequence of text. **How?**
 - multiply the probability associated with each term in the sequence.
 - $P(S|d) = \prod_{t \in S} P(t|d)$

The Retrieval Task

Input

- A corpus / collection C (set of documents : $d_1, d_2, \dots d_m$)
- A query $Q = \{q_1, q_2, \dots\}$

Output

- A set of documents relevant to the query Q .
- Boolean retrieval.
- Ranked retrieval.

Language Model for Information Retrieval

- **Task**: given a query, retrieve relevant documents.
- **Objective**: rank documents based on the probability that they are on the same topic as the query

Language Model for Information Retrieval

- **Task:** given a query, retrieve relevant documents.
- **Objective:** rank documents based on the probability that they are on the same topic as the query
- **Solution:**
 - 1 *Construct language models* associated with each of the documents of the collection.

Language Model for Information Retrieval

- **Task:** given a query, retrieve relevant documents.
- **Objective:** rank documents based on the probability that they are on the same topic as the query
- **Solution:**
 - 1 *Construct language models* associated with each of the documents of the collection.
 - 2 Given query **Q**, rank documents by the probability of the document being relevant to the query.

Language Model for Information Retrieval

- **Task:** given a query, retrieve relevant documents.
- **Objective:** rank documents based on the probability that they are on the same topic as the query
- **Solution:**
 - 1 *Construct language models* associated with each of the documents of the collection.
 - 2 Given query **Q**, rank documents by the probability of the document being relevant to the query.

$$P(D|Q)$$

Language Model for Information Retrieval

- **Task:** given a query, retrieve relevant documents.
- **Objective:** rank documents based on the probability that they are on the same topic as the query
- **Solution:**
 - 1 *Construct language models* associated with each of the documents of the collection.
 - 2 Given query **Q**, rank documents by the probability of the document being relevant to the query.

$$P(D|Q)$$

- ★ how likely that the document will be generated, given the query.

Language Model for Information Retrieval

- **Task:** given a query, retrieve relevant documents.
- **Objective:** rank documents based on the probability that they are on the same topic as the query
- **Solution:**
 - 1 *Construct language models* associated with each of the documents of the collection.
 - 2 Given query **Q**, rank documents by the probability of the document being relevant to the query.

$$P(D|Q)$$

★ how likely that the document will be generated, given the query.

- 3 *Rank the documents* based on $P(D|Q)$ in decreasing order.

Language Model for Information Retrieval

$$P(D|Q)$$

Language Model for Information Retrieval

$$P(D|Q) = \frac{P(Q|D)P(D)}{P(Q)}$$

Language Model for Information Retrieval

$$P(D|Q) = \frac{P(Q|D)P(D)}{P(Q)}$$

- $P(Q)$: same for all document $\rightarrow$ ignore
- $P(D)$: prior probability of selection of D - often treated as the same for all D .
 - ▶ But we can give a *prior* to *high-quality* documents, e.g., those with high *PageRank*.

Language Model for Information Retrieval

$$P(D|Q) = \frac{P(Q|D)P(D)}{P(Q)}$$

- $P(Q)$: same for all document $\rightarrow$ ignore
- $P(D)$: prior probability of selection of D - often treated as the same for all D .
 - ▶ But we can give a *prior* to *high-quality* documents, e.g., those with high *PageRank*.
- $P(Q|D)$: probability of Q given D .

Language Model for Information Retrieval

$$P(D|Q) = \frac{P(Q|D)P(D)}{P(Q)}$$

- $P(Q)$: same for all document $\rightarrow$ ignore
- $P(D)$: prior probability of selection of D - often treated as the same for all D .
 - ▶ But we can give a *prior* to *high-quality* documents, e.g., those with high *PageRank*.
- $P(Q|D)$: probability of Q given D .
- So, to rank documents according to relevance to $Q \rightarrow$ ranking according to $P(D|Q)$ OR $P(Q|D)$.

Language Model for Information Retrieval

$$P(D|Q) = \frac{P(Q|D)P(D)}{P(Q)}$$

- $P(Q)$: same for all document $\rightarrow$ ignore
- $P(D)$: prior probability of selection of D - often treated as the same for all D .
 - ▶ But we can give a *prior* to *high-quality* documents, e.g., those with high *PageRank*.
- $P(Q|D)$: probability of Q given D .
- So, to rank documents according to relevance to $Q \rightarrow$ ranking according to $P(D|Q)$ OR $P(Q|D)$.

Next - how to estimate $P(Q|D)$.

Estimating $P(Q|D)$

- We will make the conditional independence assumption.

$$P(Q|D) = P(\langle q_1, \dots, q_k \rangle | D) = \prod_{1 \leq i \leq k} P(q_i | D)$$

Estimating $P(Q|D)$

- We will make the conditional independence assumption.

$$P(Q|D) = P(\langle q_1, \dots, q_k \rangle | D) = \prod_{1 \leq i \leq k} P(q_i | D)$$

- This is equivalent to:

$$P(Q|D) = \prod_{q \in Q} P(q|D)^{tf(q,Q)}$$

Estimating $P(Q|D)$

- Probability of a term t in a LM M_D using Maximum Likelihood Estimation (MLE):

$$P(t|D) = \frac{tf(t, D)}{|D|}$$

Estimating $P(Q|D)$

- Probability of a term t in a LM M_D using Maximum Likelihood Estimation (MLE):

$$P(t|D) = \frac{tf(t, D)}{|D|}$$

- Probability of a query Q to be noticed in a LM D :

$$P(Q|D) = \prod_{q \in Q} \left(\frac{tf(q, D)}{|D|} \right)^{tf(q, Q)}$$

Estimating $P(Q|D)$

example



$$\begin{aligned} P(\bullet \bullet \bullet \bullet) &= P(\bullet)^2 \times P(\bullet) \times P(\bullet) \\ &= (4/9)^2 \times (2/9) \times (3/9) = 0.0146 \end{aligned}$$

Estimating $P(Q|D)$

example



$$\begin{aligned}P(\bullet \bullet \bullet \bullet) &= P(\bullet)^2 \times P(\bullet) \times P(\bullet) \\&= (4/9)^2 \times (2/9) \times (3/9) = 0.0146\end{aligned}$$

$$\begin{aligned}P(\bullet \bullet \bullet \bullet) &= P(\bullet) \times P(\bullet) \times P(\bullet) \times P(\bullet) \\&= (4/9) \times (2/9) \times (3/9) \times 0 = 0\end{aligned}$$

Estimating $P(Q|D)$

example



$$\begin{aligned}P(\bullet \bullet \bullet \bullet) &= P(\bullet)^2 \times P(\bullet) \times P(\bullet) \\&= (4/9)^2 \times (2/9) \times (3/9) = 0.0146\end{aligned}$$

$$\begin{aligned}P(\bullet \bullet \bullet \bullet) &= P(\bullet) \times P(\bullet) \times P(\bullet) \times P(\bullet) \\&= (4/9) \times (2/9) \times (3/9) \times 0 = 0\end{aligned}$$

- VSM $\rightarrow S(Q, D) = \sum_{q \in Q} w(q, Q) \cdot w(q, D) \rightarrow$ missing one term reduces score only.

Estimating $P(Q|D)$

example



$$\begin{aligned}P(\bullet \bullet \bullet) &= P(\bullet)^2 \times P(\bullet) \times P(\bullet) \\&= (4/9)^2 \times (2/9) \times (3/9) = 0.0146\end{aligned}$$

$$\begin{aligned}P(\bullet \bullet \bullet \bullet) &= P(\bullet) \times P(\bullet) \times P(\bullet) \times P(\bullet) \\&= (4/9) \times (2/9) \times (3/9) \times 0 = 0\end{aligned}$$

- VSM $\rightarrow S(Q, D) = \sum_{q \in Q} w(q, Q) \cdot w(q, D) \rightarrow$ missing one term reduces score only.

?? Language model $\rightarrow S(Q, D) = P(Q|D) = \prod_{q \in Q} P(q|D) \rightarrow$ missing one term makes score = 0.

Estimating $P(Q|D)$

example



$$\begin{aligned}P(\bullet\bullet\bullet) &= P(\bullet)^2 \times P(\bullet) \times P(\bullet) \\&= (4/9)^2 \times (2/9) \times (3/9) = 0.0146\end{aligned}$$

$$\begin{aligned}P(\bullet\bullet\bullet\bullet) &= P(\bullet) \times P(\bullet) \times P(\bullet) \times P(\bullet) \\&= (4/9) \times (2/9) \times (3/9) \times 0 = 0\end{aligned}$$

- VSM $\rightarrow S(Q, D) = \sum_{q \in Q} w(q, Q) \cdot w(q, D) \rightarrow$ missing one term reduces score only.

?? Language model $\rightarrow S(Q, D) = P(Q|D) = \prod_{q \in Q} P(q|D) \rightarrow$ missing one term makes **score = 0**.

Next $\rightarrow$ Smoothing of language model.

The Zero Frequency Problem in Language Models

The Critical Issue

$$P(Q|D) = \prod_{q \in Q} P(q|M_D) = \prod_{q \in Q} \frac{tf(q, D)}{|D|}$$

- If any query term q is missing from document D : $tf(q, D) = 0$
- Then $P(q|M_D) = 0$ and consequently $P(Q|D) = 0$
- Document gets zero score if it misses any query term!

Real Consequences

- Query: machine learning algorithms
- Document: Comprehensive machine learning textbook
- Contains machine 150 times, learning 200 times
- Uses methods instead of algorithms
- Result: $P(Q|D) = 0$ - relevant document not retrieved!

Smoothing: The Essential Solution

Core Idea of Smoothing

Reserve some probability mass for unseen words that don't appear in the document but might appear in the language.

Without Smoothing

- MLE: $P(t|D) = \frac{tf(t,D)}{|D|}$
- No probability for unseen terms

With Smoothing

- $P(t|D) = f(tf(t,D), |D|, \text{collection stats})$
- Non-zero probability for all terms

Summary: Why Smoothing is Essential

Key Benefits

- **Solves Zero Probability:** No document gets eliminated for missing one term
- **Improves Robustness:** Handles vocabulary mismatch between queries and documents
- **Better Estimation:** Accounts for data sparsity in short documents

Practical Impact

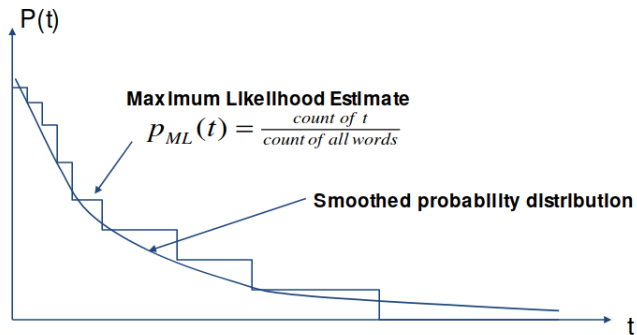
- **Recall Improvement:** More relevant documents retrieved
- **Ranking Quality:** Better discrimination between documents
- **Query Understanding:** Scope to handle synonymy and related terms

Without Smoothing

- Poor recall for multi-term queries
- Unreliable for short documents
- Vocabulary mismatch problems

Estimating $P(Q|D)$: Zero frequency

- **Problem**: Zero frequency.
- **Solution**: **Smooth** term probabilities.



Estimating $P(Q|D)$: Smoothing

- Document texts are a sample from the language model.
- Missing words should not have zero probability of occurring.
- A technique for estimating probabilities for missing (or unseen) words: **Mixture Model**.

Jelinek-Mercer Smoothing (Linear Interpolation)

$$P(t|D) = \lambda P(t|M_D) + (1 - \lambda)P(t|M_C)$$

- Mixes the probability from the document with the general collection frequency of the word.
- Estimate for **unseen** word: $(1 - \lambda)P(t|M_C)$
 - ▶ Based on **collection language model** (also called **background LM**)
 - ▶ $P(q|M_C)$: probability for query word q in the collection language model for collection C (background probability)
 - ▶ λ : parameter controlling probability for unseen words

Jelinek-Mercer Smoothing

Document Retrieval

$$score(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

- High value of λ : 'conjunctive-like' search - tends to retrieve documents containing all query words.
- Low value of λ : 'disjunctive-like' search - suitable for long queries.
- Correctly setting λ important for good performance.

Jelinek-Mercer Smoothing

The Formula

$$P(q|D) = \lambda \cdot P_{MLE}(q|D) + (1 - \lambda) \cdot P(q|C)$$

where:

- $P_{MLE}(q|D) = \frac{tf(q,D)}{|D|}$ (document model)
- $P(q|C) = \frac{cf(q)}{|C|}$ (collection model)
- $\lambda \in [0, 1]$: smoothing parameter

How It Works

- Mixes document-specific and collection-wide information
- λ controls the trade-off:
- $\lambda \rightarrow 1$: trust document more
- $\lambda \rightarrow 0$: trust collection more

Parameter Effects

- **High** λ : Conjunctive-like search
- **Low** λ : Disjunctive-like search
- Typical values: 0.1-0.7
- Tune on development data

Jelinek-Mercer Smoothing

Example

- Collection: $C = \{d_1, d_2\}$
- d_1 : Jackson was one of the most talented entertainers of all time
- d_2 : Michael Jackson anointed himself King of Pop
- Query Q : Michael Jackson

Jelinek-Mercer Smoothing

Example

- Collection: $C = \{d_1, d_2\}$
- d_1 : Jackson was one of the most talented entertainers of all time
- d_2 : Michael Jackson anointed himself King of Pop
- Query Q : Michael Jackson
- Use mixture model with $\lambda = 1/2$. Compute.

Jelinek-Mercer Smoothing

Example

- Collection: $C = \{d_1, d_2\}$
- d_1 : Jackson was one of the most talented entertainers of all time
- d_2 : Michael Jackson anointed himself King of Pop
- Query Q : Michael Jackson
- Use mixture model with $\lambda = 1/2$. Compute.
- $P(Q|d_1) = [(0/11 + 1/18)/2] \cdot [(1/11 + 2/18)/2] \approx 0.003$
- $P(Q|d_2) = [(1/7 + 1/18)/2] \cdot [(1/7 + 2/18)/2] \approx 0.013$

Jelinek-Mercer Smoothing

Example

- Collection: $C = \{d_1, d_2\}$
- d_1 : Jackson was one of the most talented entertainers of all time
- d_2 : Michael Jackson anointed himself King of Pop
- Query Q : Michael Jackson
- Use mixture model with $\lambda = 1/2$. Compute.
- $P(Q|d_1) = [(0/11 + 1/18)/2] \cdot [(1/11 + 2/18)/2] \approx 0.003$
- $P(Q|d_2) = [(1/7 + 1/18)/2] \cdot [(1/7 + 2/18)/2] \approx 0.013$
- $\text{rank}(d_2)$ lower than $\text{rank}(d_1)$.

Problem with Jelinek-Mercer Smoothing

$$score_{lmjm}(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

Problem with Jelinek-Mercer Smoothing

$$score_{lmjm}(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

- Collection: $C = \{d_1, d_2 \dots d_N\}$
- Query $Q = \{q\}$
- d_i : $|d_i| = 100$, $tf(q, d_i) = 10$
- d_j : $|d_j| = 10$, $tf(q, d_j) = 5$

Problem with Jelinek-Mercer Smoothing

$$score_{lmjm}(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

- Collection: $C = \{d_1, d_2 \dots d_N\}$
- Query $Q = \{q\}$
- d_i : $|d_i| = 100$, $tf(q, d_i) = 10$
- d_j : $|d_j| = 10$, $tf(q, d_j) = 5$
- $\text{rank}(d_j) < \text{rank}(d_i)$

Problem with Jelinek-Mercer Smoothing

$$score_{lmjm}(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

- Collection: $C = \{d_1, d_2 \dots d_N\}$
- Query $Q = \{q\}$
- d_i : $|d_i| = 100$, $tf(q, d_i) = 10$
- d_j : $|d_j| = 10$, $tf(q, d_j) = 5$
- $\text{rank}(d_j) < \text{rank}(d_i)$
- λ is same for all documents (long or short)

Problem with Jelinek-Mercer Smoothing

$$score_{lmjm}(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

- Collection: $C = \{d_1, d_2 \dots d_N\}$
- Query $Q = \{q\}$
- d_i : $|d_i| = 100$, $tf(q, d_i) = 10$
- d_j : $|d_j| = 10$, $tf(q, d_j) = 5$
- $\text{rank}(d_j) < \text{rank}(d_i)$
- λ is same for all documents (long or short)
- Shorter documents get unintended priority over long documents

Problem with Jelinek-Mercer Smoothing

$$score_{lmjm}(Q, D) = \prod_{q \in Q} P(q|D) = \prod_{q \in Q} \lambda P(q|M_D) + (1 - \lambda)P(q|M_C)$$

- Collection: $C = \{d_1, d_2 \dots d_N\}$
- Query $Q = \{q\}$
- d_i : $|d_i| = 100$, $tf(q, d_i) = 10$
- d_j : $|d_j| = 10$, $tf(q, d_j) = 5$
- $\text{rank}(d_j) < \text{rank}(d_i)$
- λ is same for all documents (long or short)
- Shorter documents get unintended priority over long documents
- **The solution:** Make the smoothing parameter dependent on the document size.

Language Model with Dirichlet Smoothing (Bayesian Approach)

$$\lambda = \frac{|D|}{|D| + \mu}$$

$$\begin{aligned} score_{lmdir}(Q, D) &= \prod_{q \in Q} \frac{|D|}{|D| + \mu} P(q|M_D) + \left(1 - \frac{|D|}{|D| + \mu}\right) P(q|M_C) \\ &= \prod_{q \in Q} \frac{|D|}{|D| + \mu} \frac{tf(q, D)}{|D|} + \left(1 - \frac{|D|}{|D| + \mu}\right) \frac{cf(q)}{|C|} \\ &= \prod_{q \in Q} \frac{tf(q, D) + \mu \frac{cf(q)}{|C|}}{|D| + \mu} \end{aligned}$$

Language Model with Dirichlet Smoothing

The Formula

$$score_{lmdir}(Q, D) = \prod_{q \in Q} \frac{tf(q, D) + \mu \frac{cf(q)}{|C|}}{|D| + \mu}$$

- $\mu > 0$: smoothing parameter (Bayesian prior strength)
- $P(q|C) = \frac{cf(q)}{|C|}$: collection language model

Key Features

- **Document-length dependent:** $\lambda = \frac{|D|}{|D| + \mu}$
- Longer documents need less smoothing
- Shorter documents need more smoothing
- More theoretically grounded

Parameter μ

- Controls smoothing strength
- Typical values: 500-2000
- Larger μ more smoothing
- Smaller μ less smoothing

Intuition

Comparing Smoothing Methods

Characteristic	Jelinek-Mercer	Dirichlet
Parameter	Fixed λ	Document-dependent $\lambda - \mu$
Length Handling	Same for all docs	Adaptive to doc length
Theoretical Basis	Linear interpolation	Bayesian prior
Typical Values	$\lambda = 0.1 - 0.7$	$\mu = 500 - 2000$
Performance	Good for known-length	Better for variable-length
Implementation	Simpler	Slightly more complex

When to Use Which

- **Jelinek-Mercer:** When documents have similar lengths
- **Dirichlet:** When document lengths vary significantly

Smoothing in Action: Example

Scenario

- Collection size: $|C| = 1,000,000$ tokens
- Term **algorithm**: $cf(\text{algorithm}) = 5,000$
- Document D : $|D| = 500$ tokens, $tf(\text{algorithm}, D) = 0$
- Query: **machine learning algorithm**

Without Smoothing

$$P(\text{algorithm}|D) = \frac{0}{500} = 0$$

$$P(Q|D) = 0$$

Document gets zero score!

With Dirichlet Smoothing ($\mu = 1000$)

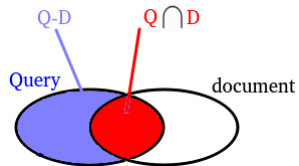
$$P(\text{algorithm}|C) = \frac{5000}{1000000} = 0.005$$

$$P(\text{algorithm}|D) = \frac{0 + 1000 \cdot 0.005}{500 + 1000} = \frac{5}{1500} \approx 0.0033$$

Non-zero probability - document can be retrieved!

Comparing Smoothing with IDF

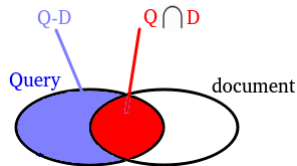
$$P(Q|D) = \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|}$$



Comparing Smoothing with IDF

$$P(Q|D) = \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|}$$

$$= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|}$$

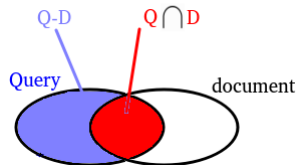


Comparing Smoothing with IDF

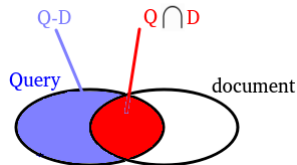
$$P(Q|D) = \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|}$$

$$= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|}$$

$$= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|}$$

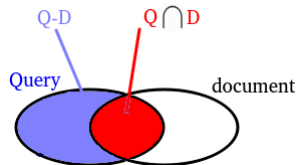


Comparing Smoothing with IDF



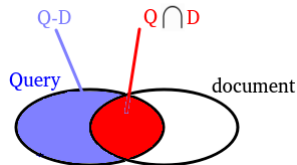
$$\begin{aligned}
 P(Q|D) &= \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}
 \end{aligned}$$

Comparing Smoothing with IDF



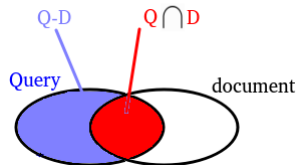
$$\begin{aligned}
 P(Q|D) &= \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{1}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}
 \end{aligned}$$

Comparing Smoothing with IDF



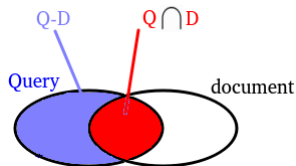
$$\begin{aligned}
 P(Q|D) &= \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{1}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{1}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}
 \end{aligned}$$

Comparing Smoothing with IDF



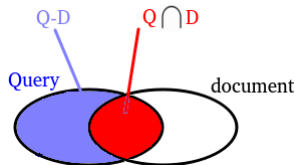
$$\begin{aligned}
 P(Q|D) &= \prod_{q \in Q} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q - D} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{1}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}} \\
 &= \prod_{q \in Q \cap D} \lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|} \times \prod_{q \in Q} (1 - \lambda) \frac{cf(q)}{|C|} \times \frac{1}{\prod_{q \in Q \cap D} (1 - \lambda) \frac{cf(q)}{|C|}} \\
 &= \prod_{q \in Q \cap D} \frac{\lambda \frac{tf(q, D)}{|D|} + (1 - \lambda) \frac{cf(q)}{|C|}}{(1 - \lambda) \frac{cf(q)}{|C|}} = \prod_{q \in Q \cap D} \left(1 + \frac{\lambda}{1 - \lambda} \frac{tf(q, D)}{|D|} \frac{|C|}{cf(q)} \right)
 \end{aligned}$$

Comparing Smoothing with IDF



$$P(Q|D) = \prod_{q \in Q \cap D} \left(1 + \frac{\lambda}{1 - \lambda} \frac{tf(q, D)}{|D|} \frac{|C|}{cf(q)} \right)$$

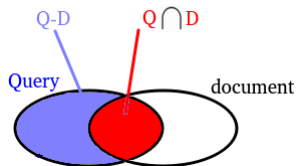
Comparing Smoothing with IDF



$$P(Q|D) = \prod_{q \in Q \cap D} \left(1 + \frac{\lambda}{1 - \lambda} \frac{tf(q, D)}{|D|} \frac{|C|}{cf(q)} \right)$$

- Rare words $\rightarrow$ less $cf(q)$ in $\frac{|C|}{cf(q)}$ $\rightarrow$ More weights

Comparing Smoothing with IDF



$$P(Q|D) = \prod_{q \in Q \cap D} \left(1 + \frac{\lambda}{1 - \lambda} \frac{tf(q, D)}{|D|} \frac{|C|}{cf(q)} \right)$$

- Rare words $\rightarrow$ less $cf(q)$ in $\frac{|C|}{cf(q)}$ $\rightarrow$ More weights
- $IDF(w) \approx \frac{|C|}{cf(q)}$

Language Models for IR: Key Takeaways

The Paradigm Shift

- Moved from **similarity-based** to **generation-based** retrieval.
- "**How likely is the query to be generated from the document?**" instead of "**How similar are query and document?**".
- Provides another **probabilistic foundation** for information retrieval.

Limitations and Legacy

- Assumes term **independence** within queries and documents.
- Sensitive to smoothing parameter choices.

Language Models vs. Traditional Approaches: Summary

Aspect	Vector Space	BM25	Language Models
Foundation	Geometric	Probabilistic + Heuristic	Probabilistic
TF Handling	Various functions	Saturation function	MLE + Smoothing
IDF	Explicit component	Explicit component	Emerges naturally
Length Norm	Cosine normalization	Parametric (b)	Smoothing-based
Parameters	Weighting schemes	k_1, b, k_3	λ, μ
Zero Score	Rare (summation)	Rare (summation)	Critical (product)
Interpretation	Similarity	Best match	Generation probability